Call center - Airflow

Проєкт симуляції роботи колл центру імітує обробку інформації про нові дзвінки. Виконується за допомогою оркестратору Airflow, який автоматично збирає та об’єднує дані.
Пайплайн в DAG бере нові дзвінки з бази MySQL, підтягує до них дані з json файлів, замінює айді працівників на їхні імена та зберігає таблицю в аналітичну базу DuckDB.

Кроки пайплайну:

1) Виявлення нових дзвінків

```
@task  1 usage
def detect_new_calls():
    last_loaded_time = Variable.get( key: "last_call_time", default_var= "2000-01-01 00:00:00")
    hook = MySqlHook(mysql_conn_id='mysql_default')
    sql = f"SELECT call_id FROM call_center.calls WHERE call_time > '{last_loaded_time}'"
    records = hook.get_records(sql)
    new_call_ids = [i[0] for i in records]
    logging.info(f"Виявлено нових дзвінків {len(new_call_ids)}")
    return new_call_ids
```

Робить запит у MySQL та знаходить так айді тільки тих дзвінків, які були додані після останнього запуску, тобто усі, які ще не оброблені до того. Збереження відбувається через вотермарку Variable last_call_time.

2) Підтягування інформації про дзвінки

```
@task  1 usage
def load_telephony_details(call_ids):
    telephony_data = []
    cnt = 0
    for j in call_ids:
        path = f"/usr/local/airflow/include/telephony/{j}.json"
        try:
            f = open(path)
            data = json.load(f)
            f.close()
            if "call_id" in data and "duration_sec" in data and "short_description" in data:
                if data["duration_sec"] >= 0:
                    telephony_data.append(data)
                else:
                    logging.warning(f"якість даних: від'ємна тривалість дзвінку у {j}.json")
                    cnt += 1
            else:
                logging.warning(f"якість даних: не всі необхідні дані заповнені у {j}.json")
                cnt += 1
        except (FileNotFoundError, json.JSONDecodeError):
            cnt += 1
            logging.warning(f"файл не знайдений, або пошкоджений {j}.json")
    logging.info(f" успішно завантажено json-файлів: {len(telephony_data)}")
    logging.warning(f" помикла/не знайдено json-файлів: {cnt}")
    return telephony_data
telephony_data = load_telephony_details(new_calls)
```

Проходиться по списку нових дзвінків з попереднього кроку (передані через XCom) та читає відповідні json файли з інформацією про дзвінки. Звідти витягує інформацію про тривалість розмови та її короткий опис.

3) Об’єднання даних в аналітичну базу

```python
@task  1 usage
def transform_and_load_duckdb(call_ids, telephony_data):
    if not call_ids:
        logging.info("Нових дзвінків не знайдено")
        return
    hook = MySqlHook(mysql_conn_id='mysql_default')
    employees = hook.get_records("SELECT employee_id, full_name FROM call_center.employees")
    emp_dict = {row[0]: row[1] for row in employees}
    ids = str(call_ids).strip('[]')
    calls = hook.get_records(f"SELECT call_id, employee_id, call_time, phone FROM call_center.calls WHERE call_id in ({ids})")
    rows_to_insert = []
    processed_ids = set()
    for c in calls:
        call_id = c[0]
        if call_id in processed_ids:
            logging.warning(f"якість даних: повторний айді дзвунка {call_id}")
            continue
        processed_ids.add(call_id)
        emp_id = c[1]
        if emp_id not in emp_dict:
            logging.warning(f"якість даних: працівника {emp_id} не знайдено для дзвінка {c[0]}")
            continue
        json_item = next((item for item in telephony_data if item["call_id"] == c[0]), None)
        combined_row = (c[0], c[2], emp_dict[c[1]], c[3], json_item['duration_sec'], json_item['short_description'])
        rows_to_insert.append(combined_row)
    conn = duckdb.connect("/usr/local/airflow/include/calls.duckdb")
    conn.execute("CREATE TABLE IF NOT EXISTS calls_final(call_id INT PRIMARY KEY, call_time TIMESTAMP, emp_name TEXT, phone TEXT,
    conn.executemany( query: "INSERT OR IGNORE INTO calls_final VALUES (?, ?, ?, ?, ?, ?)", rows_to_insert)
    logging.info(f"Нових дзвінків додано до бази {len(rows_to_insert)}")
    conn.close()
    latest_time = max(row[2] for row in calls)
    Variable.set( key: "last_call_time", latest_time.strftime("%Y-%m-%d %H:%M:%S"))
```

Об'єднує дані з MySQL та JSON в єдину структуру. На цьому етапі також підтягуються повні імена працівників. Після формування готових рядків, вони записуються у фінальну таблицю calls_final в DuckDB, а система оновлює час останнього дзвінка для наступних запусків.

Додаткові завдання:

1) Retry & alert strategy

```python
def alert_on_failure(context):  1 usage
    problem = context.get('task_instance')
    logging.warning(f"🚨 Помилка - таска {problem.task_id} впала 🚨 ")

default_args = {
    'retries': 2,
    'retry_delay': timedelta(minutes=1),
    'on_failure_callback': alert_on_failure
}
```

Двічі повторює спробу запустити пайплайн з перервою в одну хвилину, якщо не виходить - сигналізує про помилку в конкретній тасці через логи.

2) Backfill-friendly DAG (catchup, start_date, deterministic runs)

```python
with DAG(
    dag_id="enrichment_pipeline",
    default_args=default_args,
    start_date=datetime( year: 2026,  month: 3,  day: 15),
    schedule='@hourly',
    catchup = True,
    max_active_runs=1
) as dag:
```

Наздоганяє пропущені запуски (якщо вони є у розкладі щогодинних запусків), та регулює що дозволено максимум один запуск паралельно, щоб не псувати час останнього успішного запуску.

3) Observability (logging row counts, rejected JSONs)

```python
        logging.warning(f"файл не знайдений, або пошкоджений {j}.json")
logging.info(f" успішно завантажено json-файлів: {len(telephony_data)}")
logging.warning(f" помикла/не знайдено json-файлів: {cnt}")
```

Додані логи по всьому пайплайну щоб сигналізувати про успішне виконання

4) Data quality checks (duration ≥ 0, employee exists, unique call_id)

```
if data["duration_sec"] >= 0:
    telephony_data.append(data)
else:
    logging.warning(f"якість даних: від'ємна тривалість дзвінку у {j}.json")
    cnt += 1
```

Перевіряє тривалість дзвінку (вона не може бути від’ємною), сигналізує про погану якість даних

```
if emp_id not in emp_dict:
    logging.warning(f"якість даних: працівника {emp_id} не знайдено для дзвінка {c[0]}")
    continue
```

Перевіряє чи існує працівник, який відповів на дзвінок

```
if call_id in processed_ids:
    logging.warning(f"якість даних: повторний айді дзвінка {call_id}")
    continue
```

Перевіряє на повтори айді дзвінка